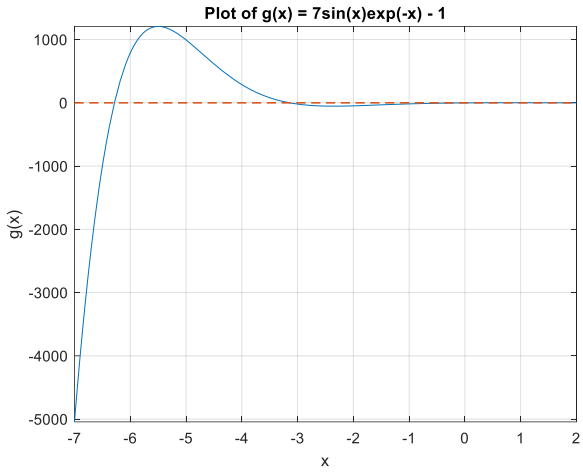
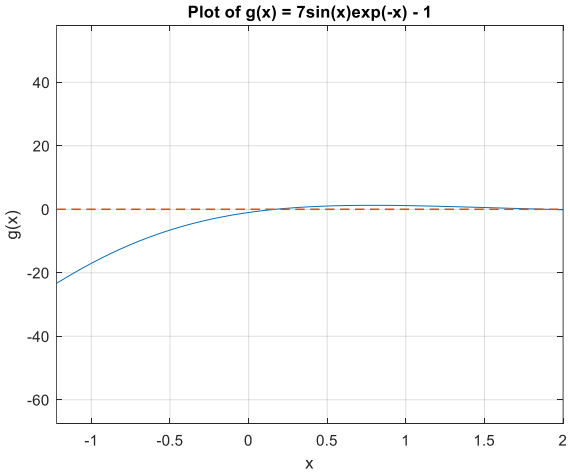
Problem 2. Contribution: equal





It seems we have 4 roots in the interval [-7,2].

For x0 = -4:

```
    "root found at:"    "-3.1477"

    "Number of iteratoins:"    "6"

Rate of convergence is:2.0006
```

For x0 = -2:

```
    "root found at:"    "0.17018"

    "Number of iteratoins:"    "6"

Rate of convergence is:1.9638
```

The rate of convergence for is nearly 2 indicating quadratic rate of convergence. The number of iterations for both are the same as well. However, choosing different initial guesses results in convergence into different roots indicating that the newton's method does not necessarily converge to the same point with different initial guesses.

Results using fzero:

```
 Func-count     x          f(x)             Procedure
      2              -2      -48.0319        initial
      3         -2.28567      -52.9757        interpolation
      4         -3.14284     -0.798254        bisection
      5         -3.14284     -0.798254        interpolation
      6          -3.1477   -0.00521876        interpolation
      7         -3.14773   2.75012e-07        interpolation
      8         -3.14773   -8.7097e-12        interpolation
      9         -3.14773   2.86438e-14        interpolation
     10         -3.14773   2.86438e-14        interpolation

Zero found in the interval [-4, -2]
>>
```

fzero used a combination of quadratic interpolation and bisection method to find the root in that interval. Results show that fzero take a couple of more iterations.

Problem 2 code:

```
%% Problem 2
% part 1, Newtons method
syms x;
g = 7*sin(x)*exp(-x) -1;
g_d = diff(g,1);

%plot the function
figure(1);
fplot(g,[-7,2]);hold on;
plot(-7:0.1:2,zeros(91,1),LineStyle="--",LineWidth=1);
xlabel("x");ylabel('g(x)');title('Plot of g(x) = 7sin(x)exp(-x) - 1');
grid on;

% implement the Newton's method
g = matlabFunction(g);
g_d = matlabFunction(g_d);
max_relative_error = 1e-6;
max_iteration = 1000;
iter = 1;
initial_guess = -2;
x = initial_guess;
x_new = x - (g(x)/g_d(x));
error_rates = [];
while (abs(x_new - x)/abs(x) >max_relative_error && iter< max_iteration )
    x= x_new;
    x_new = x - (g(x)/g_d(x));
    iter = iter + 1;
```

```
    error = x_new - x;
    error_rates = [error_rates,error];
    if(length(error_rates)>2)
        error_rates = error_rates(end-2:end);
    end
end
disp(["root found at:",num2str(x_new)]);
disp(["Number of iteratoins:",num2str(iter)]);
rate_of_convergence = log(error_rates(2))/log(error_rates(1));
disp(['Rate of convergence is:', num2str(rate_of_convergence)]);
%%
%part2: using fzero
fun = @(x) 7*sin(x)*exp(-x) -1;
x0 = [-4 -2];
options = optimset('Display','iter');
[x fval exitflag output] = fzero(fun,x0,options);
```

Problem 3; contribution: equal

I wrote newton method to solve for the exact solution (Instead of solving manually, I programmed)

The code:

```
%% Problem 3
syms x1 x2
f = 0.5*(x1^2 - x2)^2 + 0.5*(1 - x1)^2;
f_d = gradient(f,[x1,x2]);
f_dd = hessian(f,[x1,x2]);
f = matlabFunction(f,'Vars',{[x1; x2]});
f_d = matlabFunction(f_d, 'Vars', {[x1; x2]});
f_dd = matlabFunction(f_dd, 'Vars', {[x1; x2]});
initial_guess = [2;2];
max_relative_error = 1e-6;
max_iteration = 1000;
iter = 1;
x = initial_guess;
x_new = -f_dd(x)\f_d(x) + x;
step = x_new - x;

%computing values at initial value and after the first step
first_value = f(x);
second_value = f(x + step);
disp(['After 1 iteration, the new values for x1,x2 are: ',num2str(x_new(1)),', ',
num2str(x_new(2))]);
disp(['the function values after one iteration are:', num2str(first_value),',
',num2str(second_value)]);
disp("we see large decrease in the function value, so that it is a good step");
while (norm(x_new - x)/norm(x) >max_relative_error && iter< max_iteration )
    x= x_new;
    x_new = -f_dd(x)\f_d(x) + x;
    iter = iter +1;
end
```

```
disp(['Total iteration',num2str(iter)]);
```

Results:

```
After 1 iteration, the new values for x1,x2 are: 1.8, 3.2
the function values after one iteration are:2.5, 0.3208
we see large decrease in the function value, so that it is a good step
Exact minimizer: 1,1
Total iteration: 6
```

Problem 5: contribution equal

Code:

```
%% Problem 5
% Parameters
alpha0 = 1;
rho = 0.5;
c = 1e-4;
tol = 1e-6;

% Initial points
x0_1 = [1.2; 1.2];
x0_2 = [-1.2; 1];

error = 1;
iter = 0;
alpha_history_sd = [];
x = x0_2;
while error>tol
    %steepest-descent
    [f, grad] = rosenbrock(x);
    p = -grad;
    alpha = backtracking_line_search(x, p, grad, alpha0, rho, c);
    alpha_history_sd = [alpha_history_sd, alpha];
    x_new = x + alpha*p;
    error = norm(x_new -x);
    x = x_new;
    iter = iter + 1;
end
disp(["Total number of iterations for steepest_descent:",num2str(iter)]);

% Newton_method
iter = 0;
error = 1;
alpha_history_n = [];
x_n = x0_2;
while error > tol
    [~, grad, H] = rosenbrock(x_n);
    p = -H\grad;
    alpha = backtracking_line_search(x_n, p, grad, alpha0, rho, c);
```

```
    alpha_history_n = [alpha_history_n, alpha];
    x_n_new = x_n + alpha*p;
    error = norm(x_n_new - x_n);
    x_n = x_n_new;
    iter = iter + 1;
end
disp(["Total number of iterations for newton_method:",num2str(iter)]);

function [f, grad, H] = rosenbrock(x)
    a = 1; b = 100;
    f = (a - x(1))^2 + b*(x(2) - x(1)^2)^2;
% Gradient of the Rosenbrock function
    if nargout > 1
        grad = [-2*(a - x(1)) - 4*b*x(1)*(x(2) - x(1)^2);
            2*b*(x(2) - x(1)^2)];
    end
% Hessian of the Rosenbrock function
    if nargout > 2
        H = [-4*b*(x(2) - x(1)^2) + 8*b*x(1)^2 + 2, -4*b*x(1);
            -4*b*x(1), 2*b];
    end
end

% backtracking_algorithm
function alpha = backtracking_line_search(x, p, grad, alpha0, rho, c)
alpha = alpha0;
while rosenbrock(x + alpha*p) > rosenbrock(x) + c*alpha*grad'*p
    alpha = rho*alpha;
end
end
```

Results:
For first initial point (1.2,1.2):

```
    "Total number of iterations for steepe..."    "5369"

converged values for x1,x2 are: 1.0004, 1.0008
    "Total number of iterations for newton..."    "8"

converged values for x1,x2 are: 1, 1
```

For steepest descent, it takes a lot of iterations as the rate of convergence for this method is linear. Furthermore, the converged value for newton is exact, but not for steepest descent. Moreover, looking at the values of alpha (step length) shows zigzagging behavior for steepest descent while this is not the case for newton's method.

For second point (-1.2,1):

```
    "Total number of iterations for steepe..."    "6083"

converged values for x1,x2 are: 0.9996, 0.99921
    "Total number of iterations for newton..."    "22"

converged values for x1,x2 are: 1, 1
```

Step length for newton (first column) and steepest descent (second column):

22x1 double

	1
1	1
2	0.1250
3	1
4	1
5	1
6	0.2500
7	1
8	1
9	1
10	0.5000
11	1
12	1
13	1
14	0.5000
15	1
16	1
17	1
18	1
19	1
20	1
21	1
22	1

5369x1 double

	1	2
4	9.7656e-04	
5	0.0020	
6	0.0020	
7	0.0020	
8	9.7656e-04	
9	0.0078	
10	9.7656e-04	
11	0.0039	
12	0.0020	
13	9.7656e-04	
14	0.0078	
15	9.7656e-04	
16	0.0039	
17	0.0020	
18	9.7656e-04	
19	0.0078	
20	9.7656e-04	
21	0.0039	
22	0.0020	
23	9.7656e-04	
24	0.0078	
25	9.7656e-04	
26	0.0039	
27	0.0020	
28	9.7656e-04	
29	0.0078	